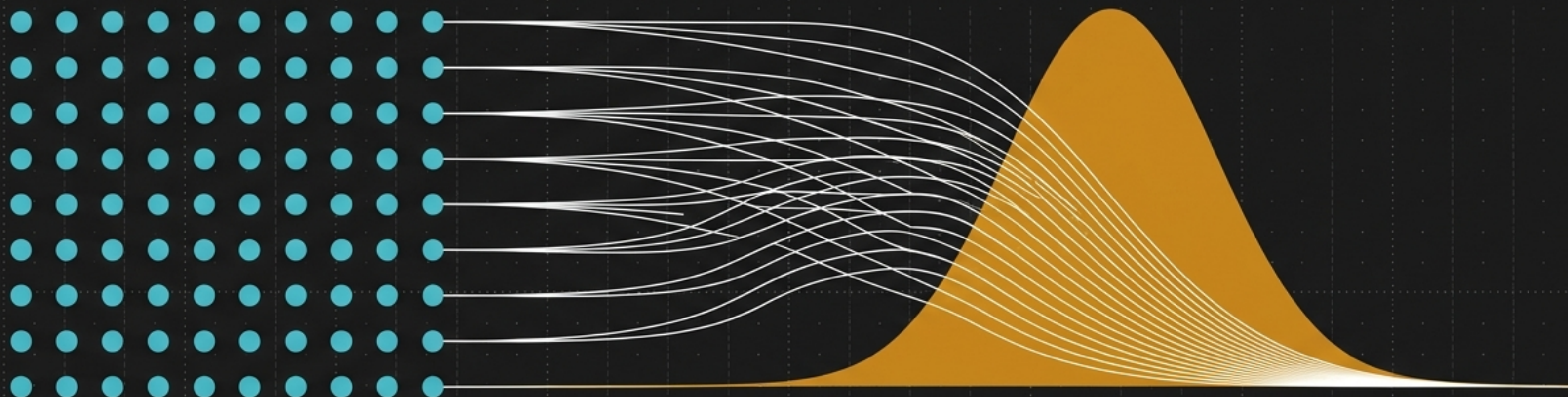
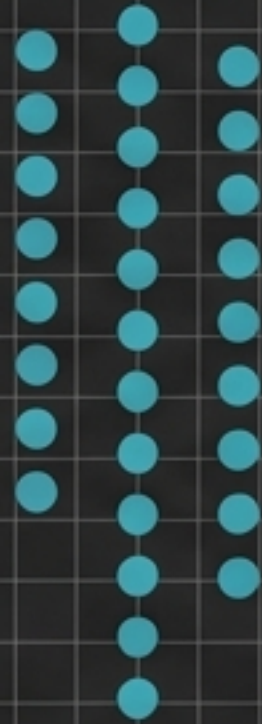




Sculpting Randomness

The Inverse Transformation Method: Theory, Proofs, and Implementation

The Raw Material



Pseudo-random number generators (PRNGs) provide a reliable, **deterministic** stream of numbers statistically indistinguishable from independent draws from a $\text{Uniform}(0,1)$.

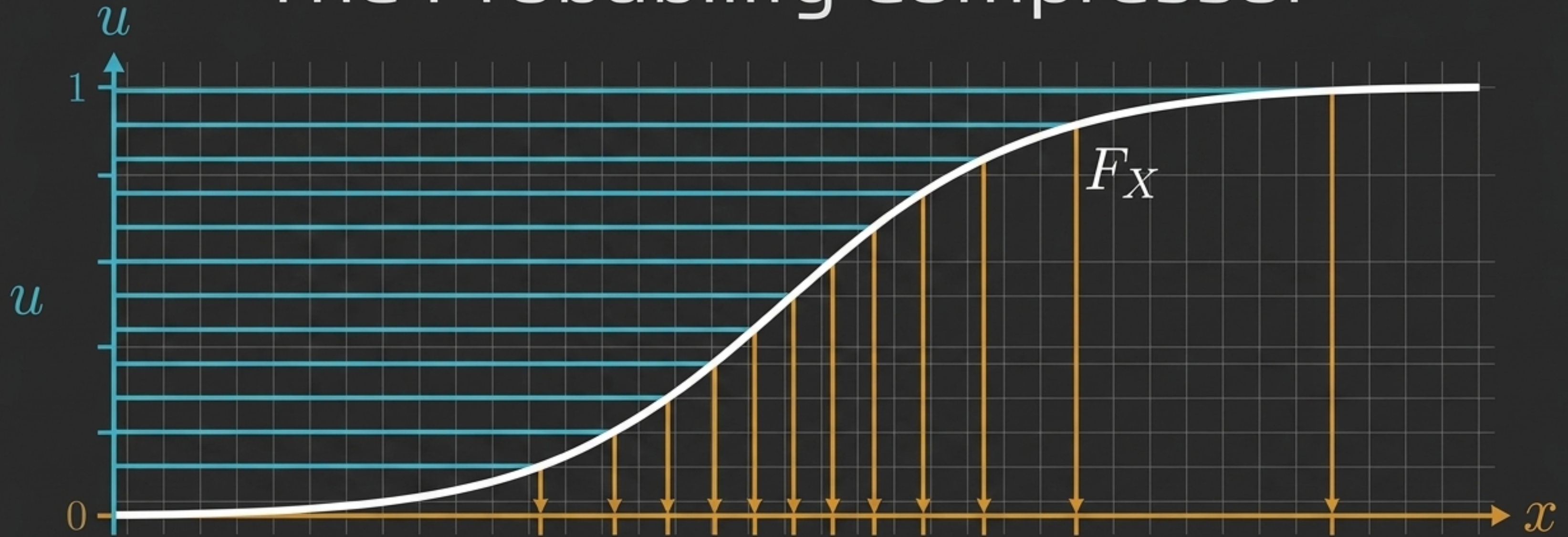
The Target Reality



Almost no simulation actually needs uniform variables. We require **target variables**: Exponential waiting times, Normal errors, or Binomial counts.

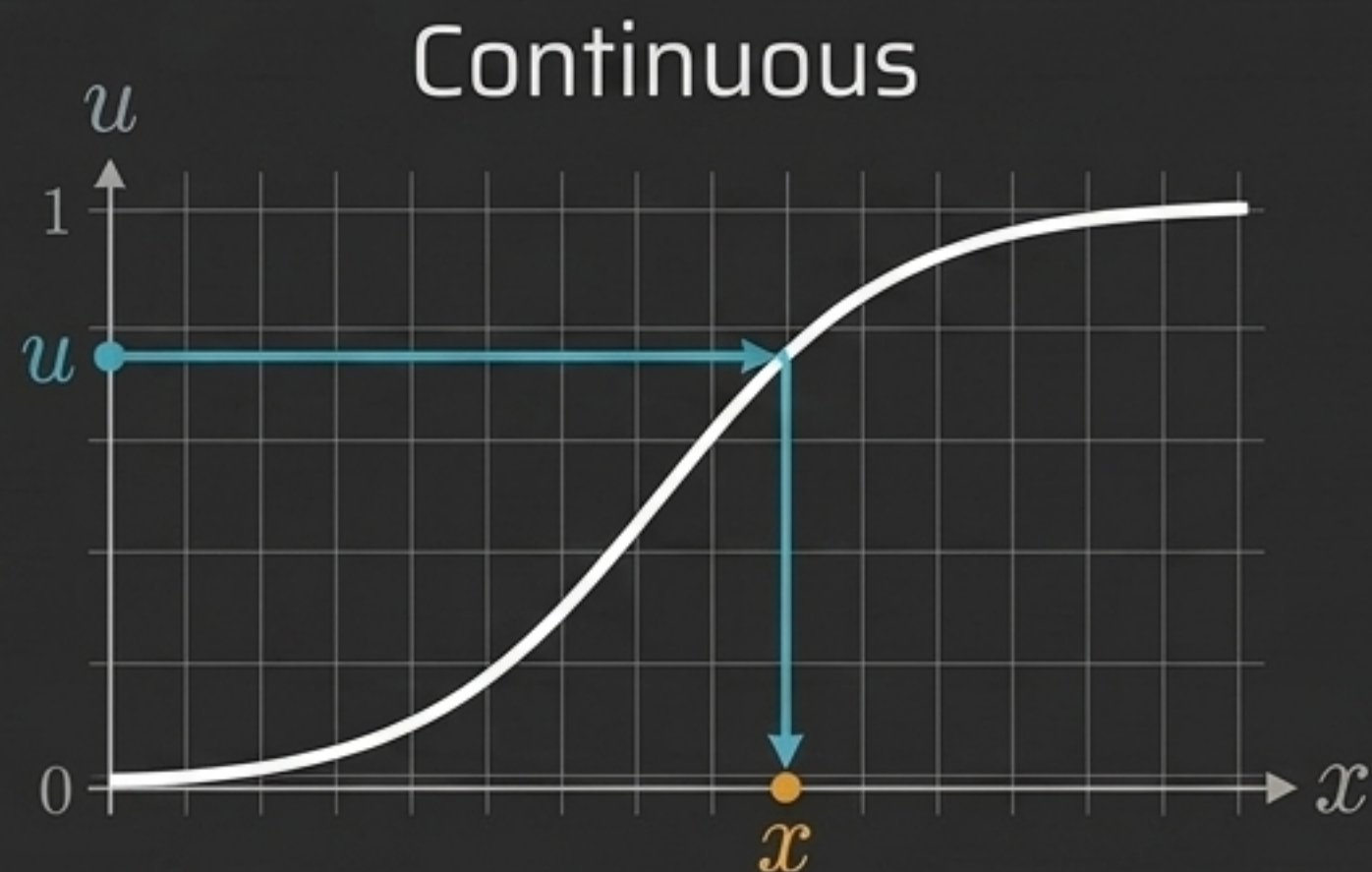
How do we map a uniform stream into any prescribed distribution using only the target's Cumulative Distribution Function (CDF)?

The Probability Compressor

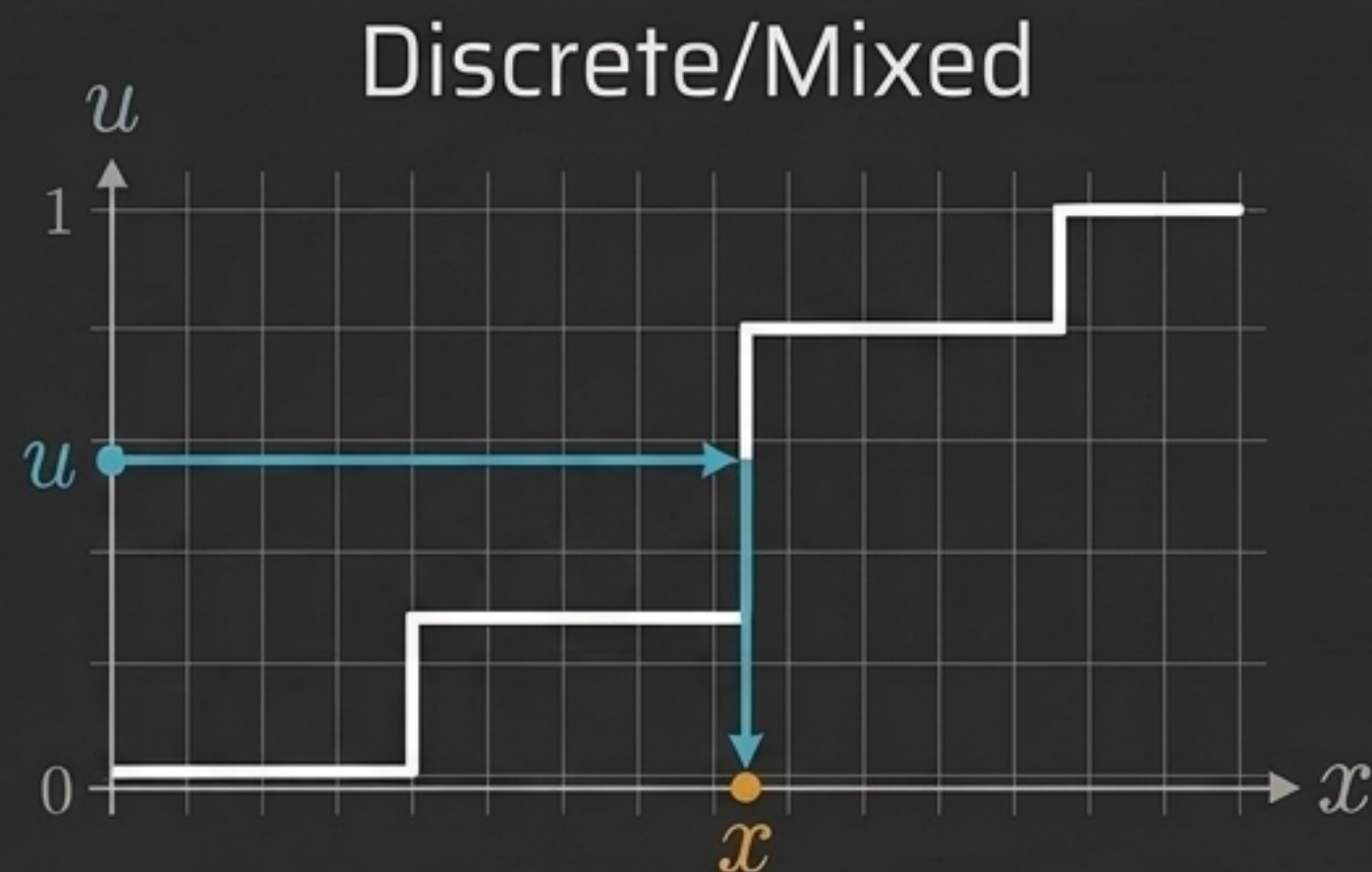


The Intuition: The **CDF** acts as a spatial compressor. A uniform sweep up the vertical axis reproduces the target distribution's **exact probability weighting** on the horizontal axis.

The Axiomatic Foundation: The Quantile Function



When F_X is continuous and strictly increasing, the standard inverse function applies perfectly.
 $u \rightarrow x$ is a 1:1 mapping.



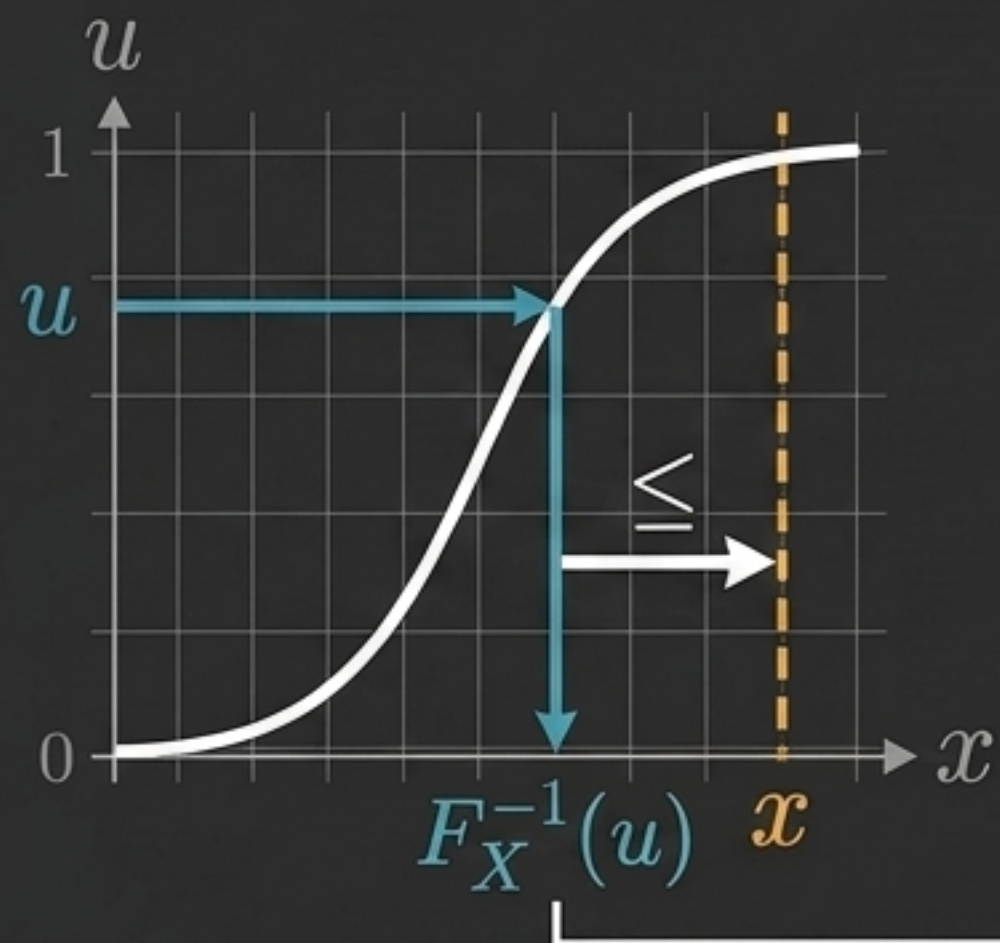
But what if the CDF has jumps or flat stretches?
We need a generalized inverse.

$$F_X^{-1}(u) = \inf\{x \in \mathbb{R} : F_X(x) \geq u\}, \quad 0 < u < 1$$

The smallest x at which the CDF has climbed up to at least height u .

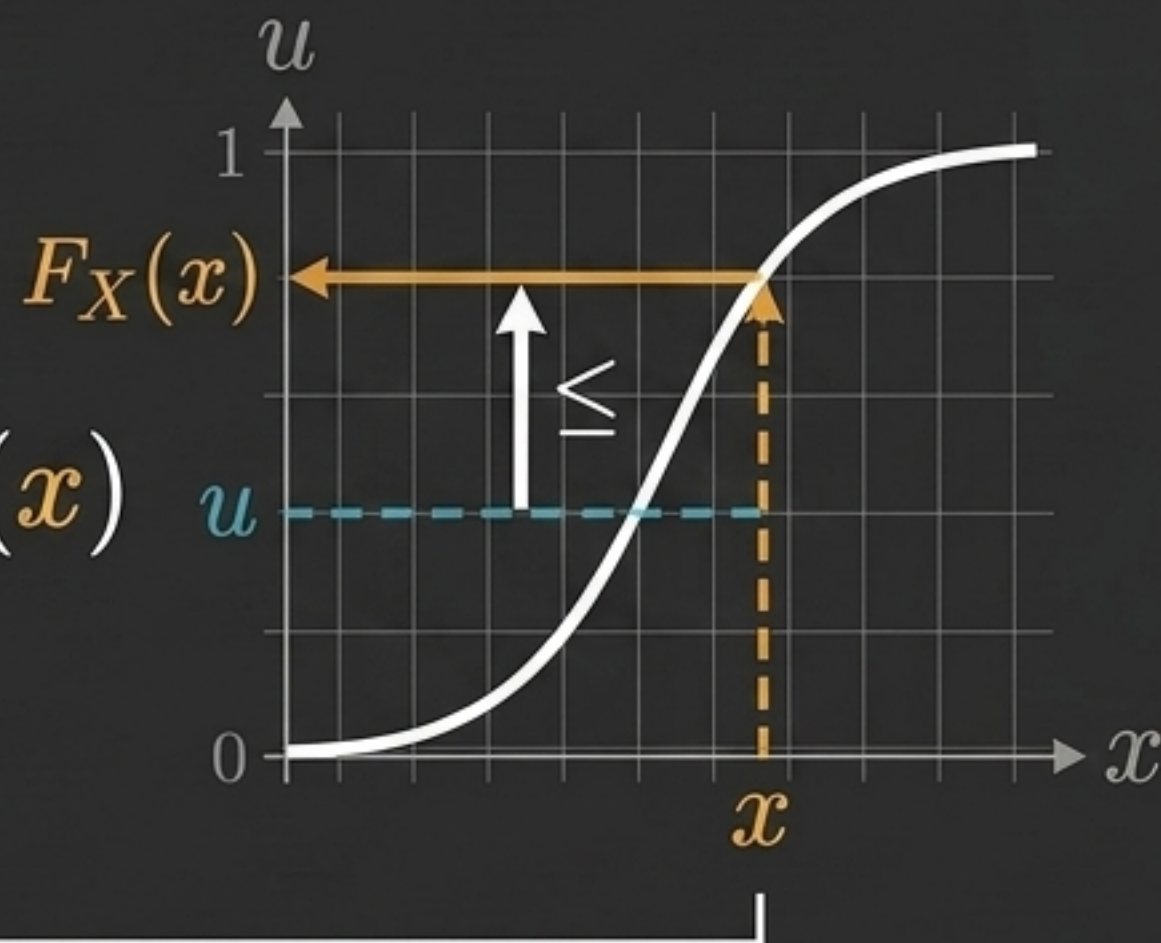
The Master Key: The Compatibility Relation

Inverse Space



$$F_X^{-1}(u) \leq x \Leftrightarrow u \leq F_X(x)$$

CDF Space



This single algebraic fact holds regardless of whether the CDF is continuous, discrete, or a mixture. It is the engine that makes the following mathematical proofs work.

Theorem 3.1: Probability Integral Transformation

Premise: If target X is continuous with cdf F_X , then $U = F_X(X) \sim \text{Uniform}(0,1)$.

Mathematical Steps	Text Justification
$P(U \leq u)$	Start with the definition of the CDF of U .
$= P(F_X(X) \leq u)$	Substitute $U = F_X(X)$.
$= P(X \leq F_X^{-1}(u))$	Apply the Compatibility Relation. 
$= F_X(F_X^{-1}(u))$	By definition of the CDF of X .
$= u$	Because F_X is continuous, applying it to its own generalized inverse returns the exact argument.

Conclusion: $P(U \leq u) = u$ is the exact CDF of a $\text{Uniform}(0,1)$ distribution.

Theorem 3.2: Inverse Transformation

Premise: If Input $U \sim \text{Uniform}(0,1)$, then Output $X = F_X^{-1}(U)$ has the exact target cdf F_X .

Logic Sequence Panel	
Mathematical Steps	Text Justification
$\mathbb{P}(X \leq x)$	Start with the CDF of our generated variable X .
$= \mathbb{P}(F_X^{-1}(U) \leq x)$	Substitute our generation mechanism $X = F_X^{-1}(U)$.
$= \mathbb{P}(U \leq F_X(x))$	Apply the Compatibility Relation. 
$= F_X(x)$	Because $U \sim \text{Uniform}(0,1)$, $P(U \leq k) = k$ for any k in $(0,1)$.

Conclusion: Since $\mathbb{P}(X \leq x) = F_X(x)$ for every x , our generated X exactly matches the target distribution. Continuity of F_X was never used—this works even if the CDF jumps.

Applied Example 1: The Exponential Distribution

Analytical Math

Target Density: $X \sim \text{Exponential}(\lambda)$

CDF: $F_X(x) = 1 - e^{-\lambda x}$

Set and Solve:

$$1 - e^{-\lambda x} = u$$

$$e^{-\lambda x} = 1 - u$$

$$F_X^{-1}(u) = -\frac{1}{\lambda} \log(1 - u)$$

Computational R Code

```
# Generate n random Uniform(0,1) draws
u <- runif(n)

# Apply the inverse CDF transformation
x <- -log(u) / lambda
```

① **Annotation:** Note that in code, we can optimize $1-u$ to just u because both share the exact same uniform distribution, saving a computation step.

Applied Example 2: Arbitrary Polynomial Density

Analytical Math

Target Density: $f(x) = 3x^2$ on $(0, 1)$

Integrate for CDF: $F_X(x) = x^3$

Set and Solve:

$$x^3 = u$$

$$F_X^{-1}(u) = u^{1/3}$$

Numeric Check: A draw of $u = 0.729$ maps to $x = 0.9$ (since $0.9^3 = 0.729$).

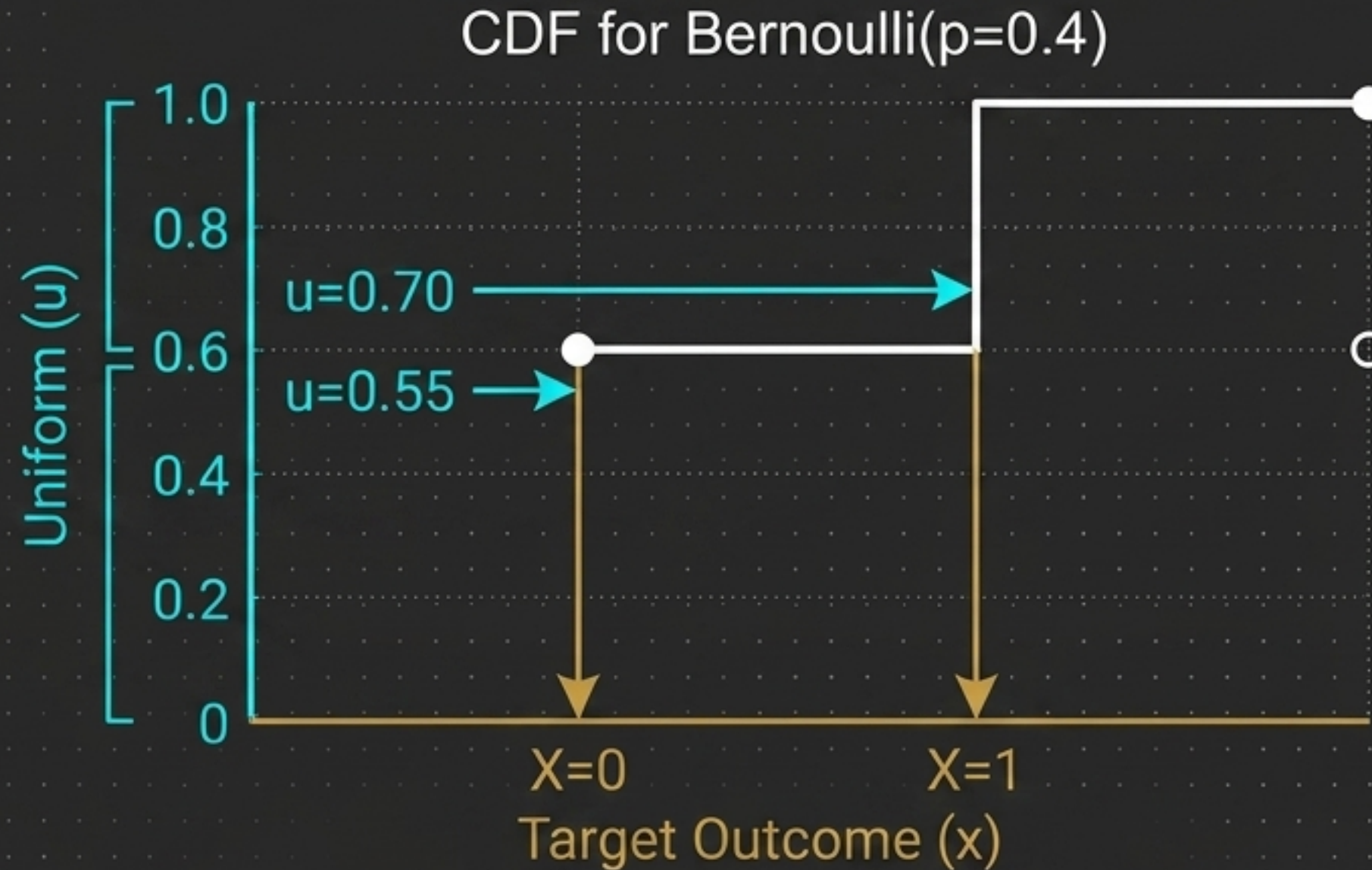
Computational R Code

```
# Vectorized R implementation  
u <- runif(n)  
x <- u^(1/3)
```

Annotation: The exact 1:1 translation from mathematical theory to one line of vectorized R code.

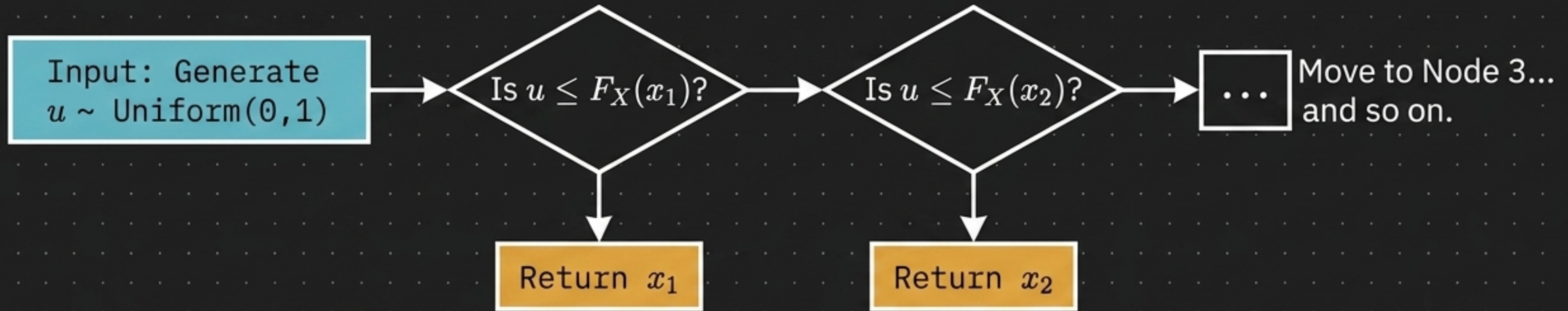
The Discrete Challenge: Searching, Not Solving

- ✓ **The Concept:** When X is discrete with support $x_1 < x_2 < x_3 \cdots$, there is no algebraic shortcut.
- ≧ **The Rule:** The compatibility relation dictates that $F_X^{-1}(u) = x_i$ whenever $F_X(x_{i-1}) < u \leq F_X(x_i)$.



Key Insight: Generating discrete variables means **searching a table of cumulative probabilities** for the first value that reaches or exceeds u .

Applied Example 3: Discrete Algorithmic Search



Real-World R Mapping:

In computational practice, instead of slow “while” loops, we generate the cumulative probability vector and evaluate logical sums:

```
x <- as.integer(sum(u > Fk))
```

(as seen in logarithmic/Poisson generators).

Synthesis: The Inverse Transformation Architecture

	Mathematical Tool	Computational Action	Speed/Efficiency	Edge Cases
Continuous Distributions	Algebraic solving ($F_X(x) = u$).	Direct vectorized formula execution (e.g., $x \leftarrow u^{1/3}$).	Extremely fast $O(1)$ operations per draw, provided the inverse exists in closed form.	Fails if the CDF cannot be analytically inverted (e.g., Normal, Gamma). Requires alternatives like Acceptance-Rejection.
Discrete Distributions	Logical inequalities ($F_X(x_{i-1}) < u \leq F_X(x_i)$).	Array searching and cumulative sum matching.	Slower $O(n)$ search time per draw; highly dependent on the number of discrete states.	Infinite support sets (like Poisson or Geometric) require dynamic generation of the CDF vector as needed to prevent memory overflow.